A | I | A | I
ADVENTURES IN AI

CASE STUDY · Live System

The Machine That Runs Itself

From one solar dashboard to thirty automated jobs running research, email triage, security, memory, and synthesis — without anyone asking them to. This is what happens when you stop doing the work and start building the system that does it.

Read the Story · Download PDF

Everyone talks about automating a task. No one talks about automating the office — the monitoring, the triage, the research, the memory, the synthesis that makes a business actually run. We didn't set out to build a self-operating system. We just kept automating the next thing that was

wasting our time. Two years later, we looked up and realised the machine was running itself.

It Started With a Solar Panel

Like a lot of good automation stories, this one started with a bill. Specifically, an electricity bill that didn't make sense. Solar panels on the roof, a battery in the garage, an export tariff that changed every half hour — and no way to see what was actually happening in real time.

The first step was a dashboard. FastAPI on the backend, HTMX for the front end, Tailwind for styling. Nothing fancy — just a page that pulled live data from the Lux Power Tek inverter API and the Octopus Energy grid API, and showed you what your solar panels were doing right now. Generation, battery state, grid import, grid export, and — critically — what it was all costing.

> ***"The moment you can see the numbers live, you start making different decisions. You don't run heavy-draw equipment at 6pm when import is 28p. You schedule it for 2pm when the panels are bursting and the battery's full."***

The dashboard worked. Then it got better. Annual projections using weather data from Open-Meteo and yield baselines from PVGIS. Live export estimates when the billing data hadn't caught up. A correction factor to account for the inverter reporting self-consumption as "export." Each addition was small. Together, they turned a monitoring screen into a decision-making tool.

Then came the widget. A macOS desktop widget — small and medium sizes, aurora gradient design — that sits on the desktop and updates every fifteen minutes. No opening a browser, no checking a tab. Just a glance and you know: panels are

generating, battery is at 87%, grid import is zero. The information is ambient. You don't have to look for it.

Then the Alerts Learned to Think

Seeing the data is one thing. Acting on it is another. The Octopus Agile tariff changes price every half hour. Sometimes prices go negative — the grid literally pays you to use electricity. But you have to know when that's happening, and you have to know in time to do something about it.

So we automated the alerts. Every thirty minutes, a scheduled job checks the Agile pricing API. If prices are negative or unusually low, it sends an iMessage: *"Prices negative at 2:30pm — run heavy-draw equipment, charge the EV, schedule high-energy processes while the grid's paying you."* If nothing's unusual, you hear nothing. The system knows the difference between information and noise.

A daily briefing follows at 4:30pm — a summary of tomorrow's pricing patterns, delivered by iMessage so it's waiting when you finish work. Again: the right information, at the right time, through the right channel. No app to check. No dashboard to open. The system pushes what you need, when you need it.

The Email Started Sorting Itself

Here's where it stopped being about solar and started being about the office. Three email accounts — personal, business, and the new Adventures in AI contact address — and a growing pile of newsletters, enquiries, spam, and the occasional genuinely important message buried in the middle of it all.

The email monitor runs every thirty minutes. It scans all three inboxes, reads every new email, and triages them into four categories:

Critical — security alerts, payment failures. Gets an immediate iMessage.

Action — questions needing a human response, meeting invites, business enquiries. Logged and batched — if three or more pile up, one summary iMessage goes out.

Info — newsletters, industry updates, anything relevant to active projects. Assessed for value, routed to the right agent, logged for the weekly synthesis.

Noise — spam, irrelevant marketing, clickbait. Marked read and forgotten.

The clever bit isn't the categorisation. It's the *value assessment*. Every Info and Action email gets evaluated: is this high-value (directly relevant to a project or opportunity), low-value (mildly interesting but no action), or just noise? High-value items get routed — to the research agent, the project board, the weekly synthesis. Low-value items get a brief note. Noise gets binned.

When an email contains a link — a newsletter pointing to an article, a shared thread — the system follows it first, reads what's on the other side, and *then* assesses value. It doesn't judge a link by its subject line. It judges it by what's actually there.

Research Without the Researcher

Every Monday, Wednesday, and Friday at 7:30am, Gav — the research agent — wakes up and produces the AI Digest. He scans a curated list of accounts on X/Twitter, checks Reddit for model releases and benchmarks, and looks for actionable developments: tools that shipped, monetisation patterns that worked, open-source releases that matter.

The output is an HTML report, delivered by email before you've even looked at your phone. Not a link dump — a curated, assessed digest with clear categories: Actionable (do something with this), Watch (keep an eye on it), and Skip (it's hype, move on). The account list itself is tiered. Tier 1 gets checked every time. Tier 2 always. Tier 3 on

Fridays only. Accounts that consistently produce actionable content get promoted. Those that don't get demoted. The research system evolves based on what works.

On Tuesdays, Thursdays, and Saturdays, a different agent — Neo — does the same thing with YouTube. Nine channels monitored, transcripts pulled for new videos, insights extracted and filed. The focus is the same: not "what happened" but "what should we do about it?"

The Sunday Synthesis

This is where it gets genuinely impressive. Every Sunday at 3am — while everyone sleeps — Bee, the coordinator agent, produces the Weekly Cross-Source Synthesis.

This isn't another digest. It's a strategic review. It reads:

- All BeeBoard pins — the living goals, ideas, and tasks that represent strategic priorities
- All active project STATUS files — what's moving, what's stuck, what changed this week
- All research from the week — YouTube insights, Gav's AI Digests, platform release radar
- All email triage — what came in, what was valuable, what got routed where
- All daily memory logs — the raw record of what actually happened
- The current system config — models, agents, providers, costs

> ***"The Sunday synthesis connects things that a human would miss because they happened in different channels on different days. A YouTube insight about agent coding patterns on Thursday connects***

> ***to a BeeBoard pin about Codex supervision from last month. That's not aggregation — that's synthesis."***

The output: a full strategic brief. Every goal reviewed for progress. Every project checked for blockers. Cross-source connections surfaced. Actions prioritised for the coming week. New goals suggested. Stale ones flagged for closure.

On Fridays at 4pm, a follow-up job checks: did we act on those recommendations? It reads the week's memory logs, compares against the Sunday actions, and presents each item with a clear choice — scope it, done already, or park it. The system doesn't just recommend. It follows up on its own recommendations.

The Infrastructure That Nobody Thinks About

Underneath all the visible automation, a layer of infrastructure keeps everything running:

Memory dreaming — three phases every night at 3am. The system reviews recent conversations, promotes important patterns to long-term memory, and makes creative connections between seemingly unrelated events. Light, Deep, and REM phases, each doing different work. This is how an AI system remembers what matters without hoarding everything.

iMessage health checks — every four hours, the system pings the iMessage bridge. If it detects XPC degradation (a known macOS issue where the messaging process silently breaks), it kills and restarts the service. Nobody notices because it's fixed before it matters.

Security scans — weekly bloat audit and security review. Disk usage, stale sessions, model cleanup, permission checks.

Log rotation — daily, keeps seven compressed backups, maximum 10MB each. Because logs grow forever if you let them.

Platform release radar — every Monday, scan for new Ollama models, OpenClaw updates, provider price changes, and new tools. Categorised as Act Now, Consider, or Aware. Specific recommendations, not vague awareness.

Git crawl sync — every three hours, sync repository state across all active projects. New issues, new PRs, status changes — all captured without anyone checking manually.

None of this is glamorous. It's the digital equivalent of checking the boiler pressure and testing the smoke alarms. But without it, the visible automation degrades. The email monitor misses things because iMessage broke. The synthesis runs on stale data because the repos didn't sync. The widget shows yesterday's numbers because no one rotated the logs and the disk filled up.

Infrastructure is the difference between a demo and a system.

The Lark Watcher

A practical example of how this scales to business needs. Many businesses use Lark (formerly Feishu) for internal messaging. It's not something OpenClaw natively integrates with — there's no API key, no webhook, no plugin.

So three times a day — 8am, 12pm, 5pm on weekdays — a scheduled job opens a browser, navigates to Lark Messenger, reads the screen, identifies any new or unread messages, and sends a summary via email. If there's nothing new, you hear nothing. If the session has expired and needs re-authentication, it sends an immediate iMessage alert — because a broken authentication is more urgent than a missed message.

This is automation in the real world. Not everything has an API. Not everything has a webhook. Sometimes you need a browser, a schedule, and the judgment to know the difference between "nothing happened" and "something broke."

Even the Appointment Watch

Anyone who's used an online GP booking system knows the frustration: you log in, check for appointments, find nothing available, log out, try again tomorrow. The slots appear and disappear in windows so narrow that human checking cadences almost always miss them. You need to see a specific doctor. The system only shows you what's available right now. So you keep checking.

This is exactly the kind of repetitive watching that humans are bad at and machines are good at. So we automated it. Three times a day, the system logs into the booking portal, navigates to the appointments page, and checks for available slots with a specific clinician. If one appears, an alert goes out. If only other clinicians are available, no message — the requirement was specific. If no appointments are available at all, no message. The system doesn't book. It doesn't access patient records. It watches and alerts, with the exact specificity that was asked for.

> ***"The appointment watch is the thing that makes people go quiet when I describe the system. It's not impressive because it's technically hard. It's impressive because it's the kind of thing nobody thinks to automate — and then you realise how much of your day is spent doing exactly this kind of watching and waiting."***

The Full Inventory

Thirty scheduled jobs. Here's what's actually running, right now, on this system:

- **SOLAR & ENERGY**
  - Widget data update — every 15 min
  - Agile live price check — every 30 min

Agile daily briefing — 4:30pm daily

Annual projection refresh — continuous

COMMUNICATION

Email triage (3 accounts) — every 30 min

Lark Messenger check — 3× daily weekdays

iMessage health check — every 4 hours

Online appointment availability check — 3× daily

RESEARCH & INTELLIGENCE

AI Digest (X + Reddit) — MWF 7:30am

YouTube channel monitor — T/Th/Sat 9am

Weekly YouTube catch-up — Mondays 10am

Platform release radar — Mondays 3am

SYNTHESIS & STRATEGY

Sunday cross-source synthesis — Sun 3am

Friday action follow-up — Fri 4pm

Weekly bloat audit + security scan

Model override cleanup — every 6 hours

INFRASTRUCTURE

Memory dreaming (3 phases) — daily 3am

Log rotation — daily 4am

Session archiving — weekly

Git crawl sync — every 3 hours

Wiki frontmatter maintenance — daily 3:30am

OpenClaw update check — daily 9:05am

- **SPECIAL**

  Model availability watch — weekly

  Smart Energy Optimiser alerts

  BeeBoard pin extraction for synthesis

  Custom one-shot reminders

The Architecture Principle

There's a pattern here that matters more than any individual job. Every automation in this system follows the same principle:

Push, don't pull.

Traditional software makes you go look for information — open a dashboard, check an inbox, refresh a page. This system pushes. The solar widget is on the desktop. The Agile alert arrives by iMessage. The email triage happens silently and only surfaces what matters. The research digest arrives by email before you're awake. The Sunday synthesis connects dots you didn't know were dots.

The user doesn't check the system. The system checks itself, and only speaks when it has something to say.

That's the architectural difference between a collection of scripts and a self-operating office. Scripts wait to be run. A self-operating system runs itself and reports by exception.

What We Learned Building It

1

Start with what's annoying you

Nobody sat down and designed a "self-operating office." We automated the electricity bill. Then the email. Then the research. Each step was small and obvious. The system emerged from the sequence, not from a plan.

2

Push beats pull every time

If your automation still requires someone to check it, it's not automated — it's just faster. The real shift happens when the system comes to you: alerts for what's urgent, silence for what isn't. Design for the recipient, not the data.

3

Infrastructure is the difference between demo and production

The email monitor, the synthesis, the research digest — none of it works if iMessage broke yesterday and no one noticed. Health checks, log rotation, session cleanup: invisible work that keeps visible work running. If you're not automating the maintenance, you're not done.

4

Value assessment is more important than categorisation

Sorting email into "important" and "not important" is table stakes. The real value is asking: what happens next with this? Does it connect to a project? Does it change a decision? Does it just sit there? Route the high-value stuff. Note the low-value stuff. Bin the noise. The system should have opinions about what matters.

5

Synthesis is the crown jewel

Any system can collect information. The Sunday synthesis is different because it connects information. A YouTube insight on Thursday and an email enquiry on Tuesday and a BeeBoard goal from last month — separately, they're data points. Connected, they're a strategy. The highest-value automation is the one that makes connections a human would miss.

6

Not everything has an API — and that's okay

The Lark Messenger check and the GP appointment check don't use APIs. They use a browser, a schedule, and judgment. In the real world, most business systems aren't API-first. If your automation only works when there's a clean integration, it only works in demos. Build for the world you actually live in.

What This Means for Your Business

You don't need thirty automated jobs. You probably need three or four — the ones that are eating your time right now. The email you check obsessively. The pricing you should be tracking. The competitor moves you keep missing because you were busy with the email.

But the principle scales. Start with the annoyance. Automate the next obvious thing. Let the system push instead of making you pull. Build the infrastructure that keeps it running. And when you look up, you'll find that the machine runs itself — and you're free to do the work that actually needs a human.

We can help you get there. Whether you want us to build it with you, or you want to build it yourself and just need someone who's already made the mistakes — we've been where you are, and we know the path from "one annoying task" to "a system that works while you sleep."

Ready to Stop Checking and Start Building?

Whether you want a self-operating office or just want to stop checking your email every ten minutes, we can help. From a single automation to a full system — we've built it, we run it, and we can show you how.

Talk to Us | More Case Studies